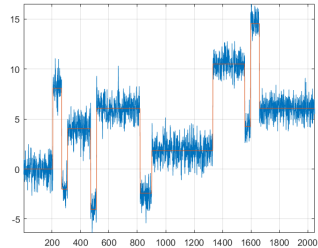
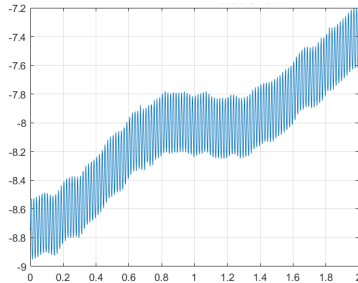
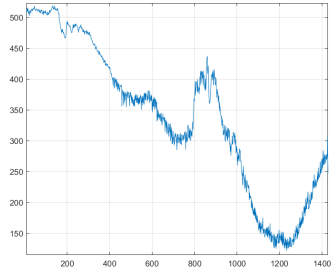
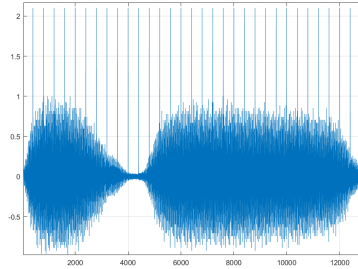




# Nettoyage des signaux Filtrage, Débruitage, Lissage, Outlier

Jean-Marc Girault  
*[jean-marc.girault@eseo.fr](mailto:jean-marc.girault@eseo.fr)*



L'extraction d'information (features) à partir de tendances, cycles, bruits issus de signaux corrompus doivent faire l'objet de traitements <sup>1</sup> du types :

- a) Filtrage fréquentiel (coupe-bande 60 Hz par exemple);
- b) Lissage / Extraction de tendance (MA, Median, Stavisky-Golay);
- c) Élimination des outliers (Hampel);
- d) Filtrage/Lissage via décomposition (ondelette);
- e) Filtrage optimal de Wiener;
- x) Les Bonus.

Les techniques doivent être adaptées en fonctions des signaux étudiés, des types de bruit et outliers.

---

<sup>1</sup>Les méthodes d'apprentissage ne seront pas traitées ici.

- Concept : décomposition atomique
- Concept : Orthogonalisation
- Article : Orthogonalisation
- Concept : Filtrage numérique
- Concept : Méthodes algébriques
- Concept : méthodes itératives

## a) Filtrage fréquentiel

## a) Filtrage :

Soit un signal évoluant très lentement, corrompu par une interférence de 60 Hz.

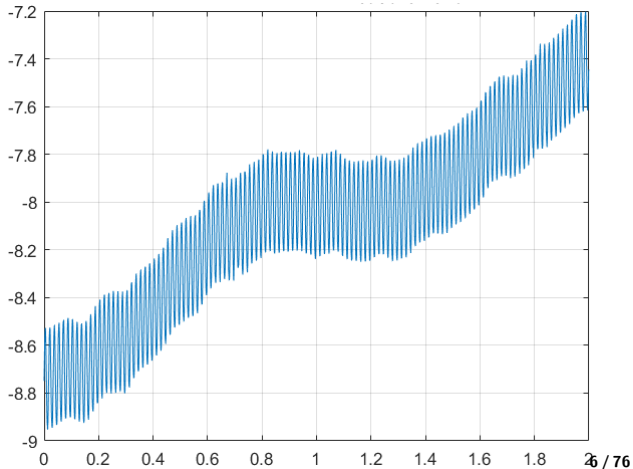
L'information à garder est la tendance.

Deux stratégies s'opposent :

- Extraction de l'interférence puis soustraction ;
- Extraction directe de la tendance;

avec des méthodes fréquentielles ou temporelles :

- Filtre coupe-bande / notch filter avec des filtres RIF ou RII;
- Méthode à base de détection d'enveloppe;
- Méthode à base de modèles (SARIMA voir autre cours);



## a) Filtrage : notch filter

**Problème :** Extraire la tendance en éliminant l'interférence de 60 Hz.

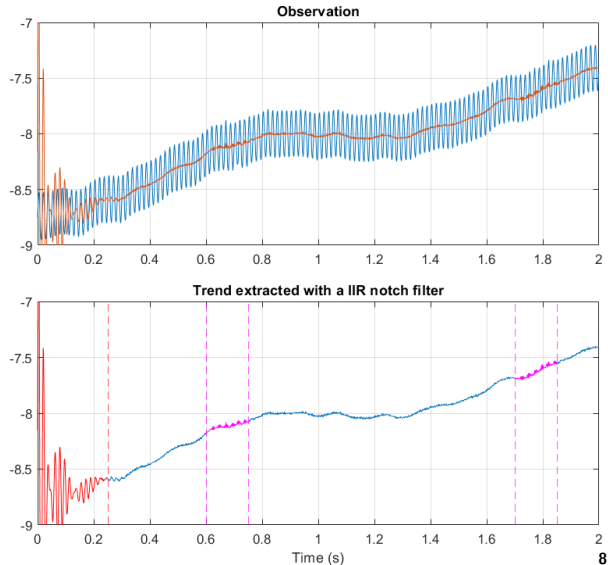
**Solution :** Filtre coupe-bande / notch filter avec des filtres FIR ou IIR.

```
clear;clc;close all;
load openloop60hertz.mat
Fpass1 = 50;Fstop1 = 59;Fstop2 = 61;Fpass2 = 70;
Apass1 = 0.5;Astop = 30;Apass2 = 0.5;Fs = 1e3;
NotchFilter = designfilt("bandstopiir", PassbandFrequency1=Fpass1, ...
    StopbandFrequency1=Fstop1, StopbandFrequency2=Fstop2, ...
    PassbandFrequency2=Fpass2, PassbandRipple1=Apass1, ...
    StopbandAttenuation=Astop, PassbandRipple2=Apass2, ...
    DesignMethod="butter", SampleRate=fs);
%filterAnalyzer(NotchFilter)
dataIn = signal;dataOut = filter(NotchFilter,dataIn);
subplot(211);plot(t,signal,t,dataOut);
subplot(212);plot(t,dataOut,t,dataOut,t(1:250),dataOut(1:250));
```

## a) Filtrage : notch filter

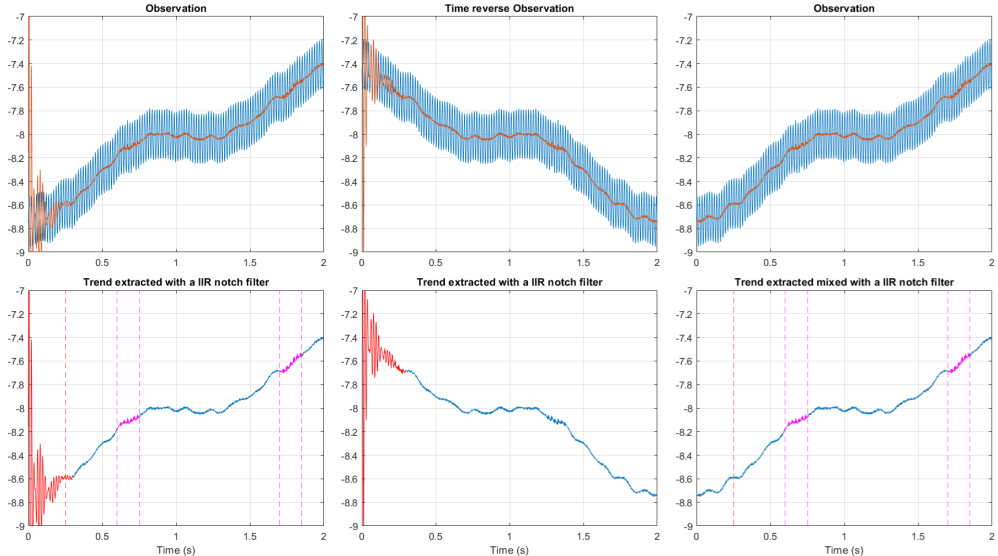
### Observations :

- Génération de grandes oscillations décroissantes en début de signal filtré (partie rouge);
- Génération de petites oscillations dans signal filtré (parties magenta);
- Les différents réglages peuvent amener un offset;
- Filtrage RIF pas adapté ici.





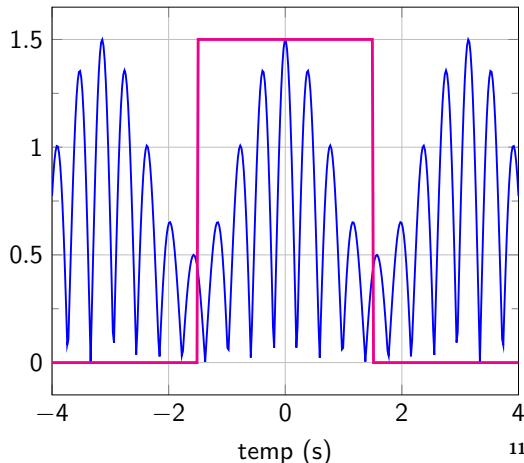
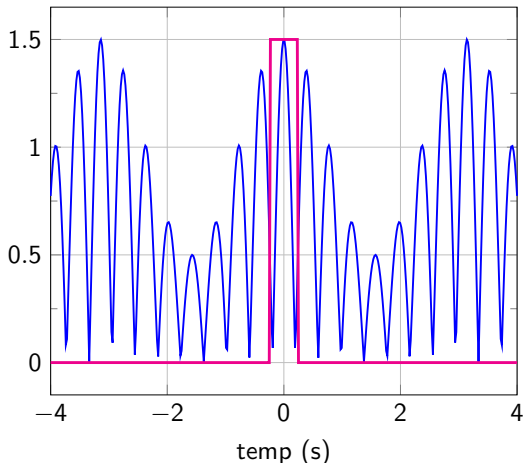
## a) Filtrage : notch filter (amélioration)



## b) Lissage

## b) Lissage : Moving Average Filter

Soit  $y(t) = (1 + \cos(\omega_m t)) \times \cos(\omega_p t)$ . Il s'agit par exemple de calculer la moyenne d'un signal rectifié dans une fenêtre glissante :  $m(\tau) = \frac{1}{L} \int_{-L/2}^{L/2} |x(t)| \text{Rect}_L(t - \tau) dt$ .



## b) Lissage : Moving Average Filter

Le réglage de la largeur  $L$  de la fenêtre est très important. Plusieurs résultats différents pour différentes valeurs de  $L$  avec  $y(t) = Env(t) \times \cos(\omega_p t)$  et avec  $Env(t) = (1 + \cos(\omega_m t))$ .

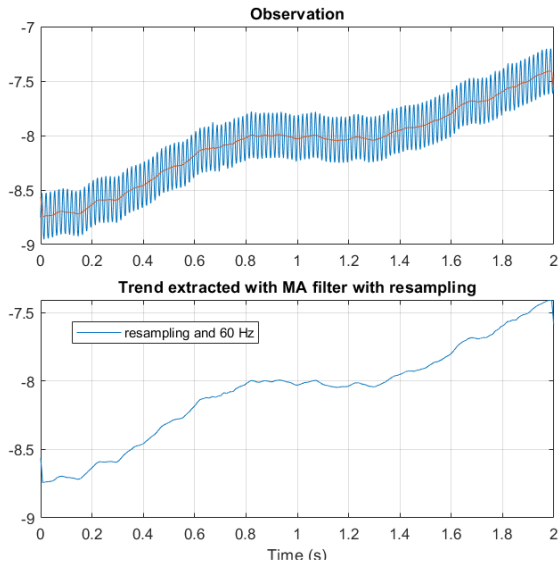
- Pour  $L = T_p/2$  (et non pas  $T_p$  car signal rectifié) alors  $m(\tau) = g(Env(t))$ ;
- Pour  $L = T_m/2$ , on obtient  $m(t) = cst$ .

```
clear;clc;close all
N=1000;Duration=8;
t=0:Duration/(N-1):Duration;
fm=1;fp=8;a=(1+0.5*cos(2*pi*fm*t));
p=cos(2*pi*fp*t);y=a.*p;%+trend;
for k=1:2:256
    plot(t,abs(y),t,movmean(abs(y),k))
    pause(0.1);
end
```

## b) Lissage : Moving Average Filter

### Observations :

- Il y a 17 échantillons ( $1000 / 60 = 16.67$ ) dans un cycle complet de 60 Hz, ce qui donne une fréquence  $1000 \text{ Hz} / 17 = 58.82 \text{ Hz}$  et non pas de 60 Hz !
- Ré-échantillonnage à la fréquence :  $17 \times 60 \text{ Hz} = 1020 \text{ Hz}$ ;
- Très bonne extraction de la tendance.



**Problème :** Extraire la tendance en éliminant l'interférence de 60 Hz.

**Solution :** Filtre à moyenne mobile avec sur-échantillonnage.

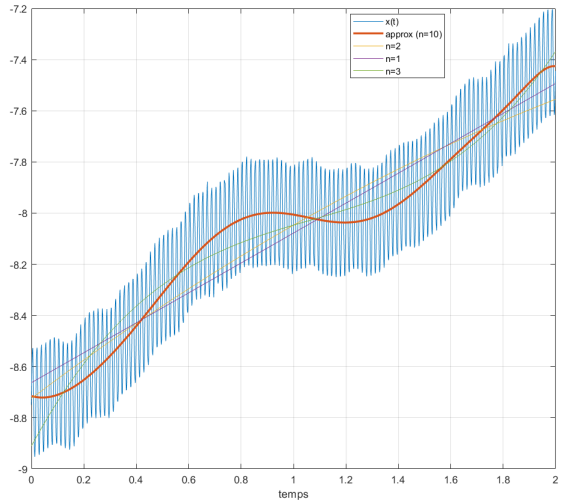
```
clear;clc;close all;
load openloop60hertz.mat
fsResamp = 1020; % 17 x 60 Hz = 1020 Hz
vResamp = resample(signal, fsResamp, fs);
tResamp = (0:numel(vResamp)-1) / fsResamp;
vAvgResamp = sgolayfilt(vResamp,1,17);
vResamp2 = resample(vAvgResamp, fs,fsResamp);

subplot(211);plot(t,signal,tResamp,vAvgResamp);
subplot(212);plot(tResamp,vAvgResamp);
```

## b) Lissage par Interpolation :

### Interpolation polynomiale.

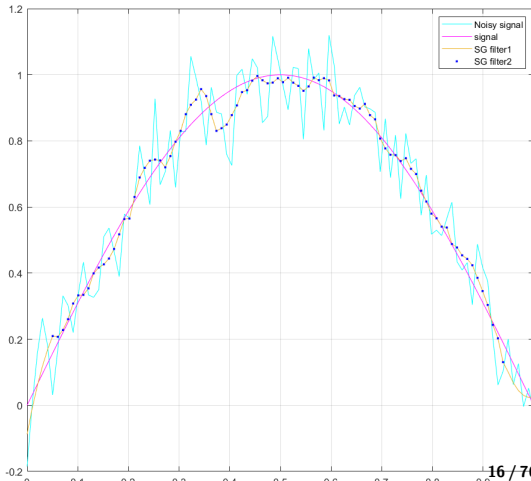
```
clear;close all;clc
load openloop60hertz.mat
N=length(signal);
order=10;
p = polyfit(t,signal,order);
y = polyval(p,t);
plot(t,signal,t,y)
```



## b) Lissage : Savitsky-Golay

Filtre utilisant une fenêtre glissante dans laquelle une interpolation est réalisée. De façon courante, l'interpolation est un polynôme d'ordre 2 dont seul le point milieu est gardé.

```
clear;close all;clc
N=100;Duration=1;
t=(0:Duration/(N-1):Duration)';
x=sin(2*pi*1/2*t);b=0.1*randn(N,1);
y=x+b;L=11;order=2;
z = smoothdata(y,"sgolay",L);
for k=1:N-L
    tL=k:k+L-1;yL=y(tL);
    pL = polyfit(tL,yL,order);
    yLest = polyval(pL,tL);
    yest(k)=yLest(ceil(L/2));
end
plot(t,y,'c',t,x,'m',t,z);hold on;
plot(t(1:N-L+ceil(L/2)-1), ...
    [NaN*ones(1,ceil(L/2)-1) yest],'b.')
```





## b) Lissage : Savitsky-Golay

Le filtre SG est défini par 2 paramètres :

- $m$  : l'ordre du polynôme;
- $n_h$  : le demi-intervalle de la fenêtre glissante (intervalle :  $2 \times n_h + 1$ ).

On peut approximer la fréquence de coupure à -3 dB à partir de la formule donnée par Schmid et al. lorsque  $n_h \geq 25$  par :

$$F_c = \frac{F_s}{6.352 \frac{(n_h+1/2)}{(m+1.379)} - \frac{(0.513+0.316 m)}{n_h+1/2}}.$$

En fixant la fréquence de coupure, on peut approximer le demi-intervalle par :

$$n_h = \frac{(0.509 + 0.1922 m - 0.001485 m^2)}{F_c/F_s} - 1$$

```
m = 6;nh = 50;f1 = 2*nh+1;
Fs = 1000;Fc = Fs/(6.352*(nh+1/2)/(m+1.379) - (0.513+0.316*m)/(nh+1/2));
disp(Fc/Fs)
nhest = round((0.509+0.1922*m-0.001485*m*m)/(F3dB/Fs)-1);
disp(table([nh;nhest],VariableNames="Value",RowNames=["nh";"nhest"]))
```

On peut calculer la fonction de transfert.

```
m = 6;nh = 50;f1 = 2*nh+1;
[u,gSG] = sgolay(m,f1);gSG = gSG(:,1);
NDFT = 1024;[HSG,F] = freqz(gSG,1,NDFT,Fs);
plot(F/Fs,mag2db(abs(HSG)))
```

On peut aussi intégrer une fenêtre d'apodisation.

```
pspectrum([s sgolayfilt(s,3,301) sgolayfilt(s,3,301,hamming(301))],Fs)
legend(["Input" "Output, no weights" "Output, with weights"])
xlim([0 0.1*Fs/2])
```

## b) Lissage : enveloppe moyenne

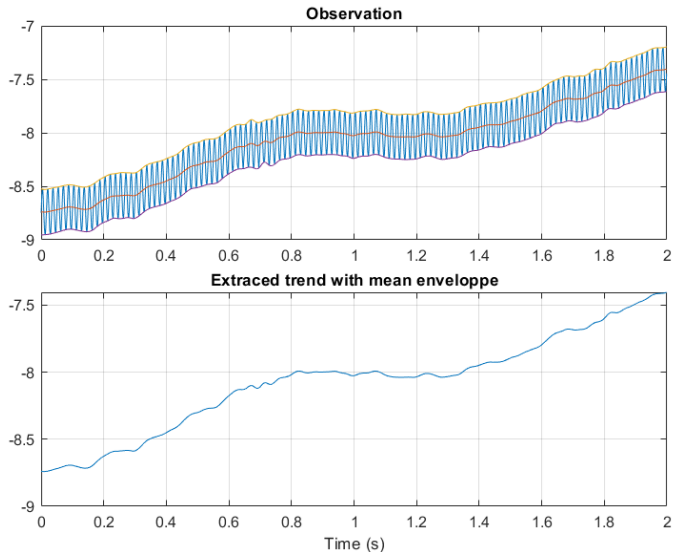
Après observation du signal, on peut faire l'hypothèse suivante :

$$x(t) = a(t) + \sin(\omega_0 t).$$

On souhaite extraire la tendance  $a(t)$  via une fonction

$$Trend[x(t)] = \frac{Env^+ + Env^-}{2} = a(t).$$

Les calculs de  $Env^+$  et  $Env^-$  ne doivent pas être réalisés avec la méthode analytique car dans ce cas  $Env^+ = Env^-$  et  $Trend[x(t)] = 0$ .



## b) Lissage : enveloppe moyenne

**Problème** : Extraire la tendance en éliminant l'interférence de 60 Hz.

**Solution** : Estimation de l'enveloppe moyenne.

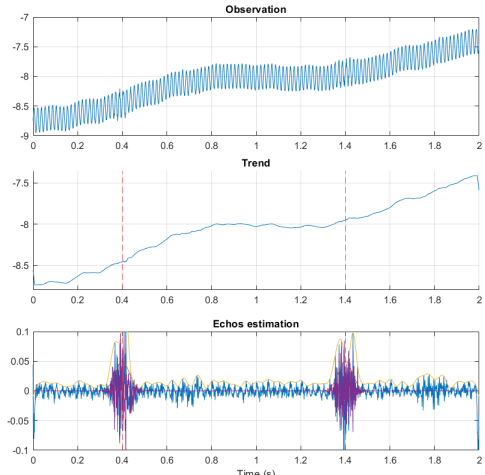
```
clear;clc;close all;
load openloop60hertz.mat
np=5;
%np: Peak separation, np<17
[yupper,ylower] = envelope(signal,np,'peak');
trend_env= (ylower+yupper)/2;
subplot(211);plot(t,signal,t,trend_env,t,yupper,t,ylower);
subplot(212);plot(t,trend_env);
```

**Remarque** :  $Env^+$  et  $Env^-$  sont estimés à partir de l'estimation des pics, ce qui assure  $Env^+ \neq Env^-$ .

Deux petits échos sont corrompus par un signal de tendance (la température par exemple) et une interférence de 60 Hz. L'information à extraire sont les 2 échos.

### Méthode 1 :

- Estimation puis extraction de la tendance ;
- Isolation du signal oscillant;
- Pas de problèmes d'oscillations;
- Estimation de l'enveloppe centrée sur signal oscillant.

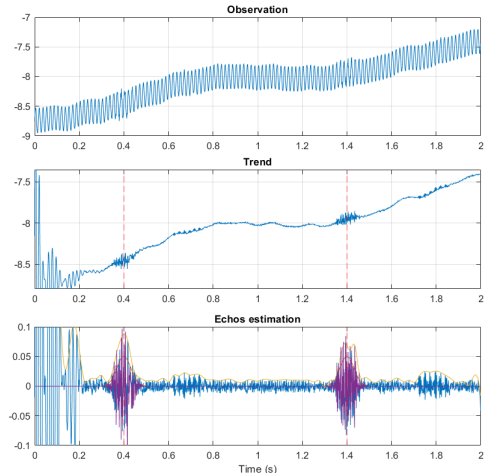


## b) Lissage :

**Deux petits échos** sont corrompus par un signal de tendance (la température par exemple) et une interférence de 60 Hz. L'information à extraire sont les 2 échos.

### Méthode 2 :

- Isolation du signal oscillant (notch filter);
- Estimation puis extraction de la tendance ;
- Estimation de l'enveloppe centrée sur signal oscillant.



## c) Élimination des outliers

## c) Élimination des outliers

**Filtre de Hampel** : similaire à la recherche de variations supérieures à  $3 \times \sigma$ . Dans une fenêtre glissante, le filtre identifie les points  $x_k$  vérifiant  $|x_k - \gamma_k| > \eta \sigma_k$  comme des outliers,  $\eta$  étant un seuil à définir (par exemple,  $\eta = 3\sigma$ ).

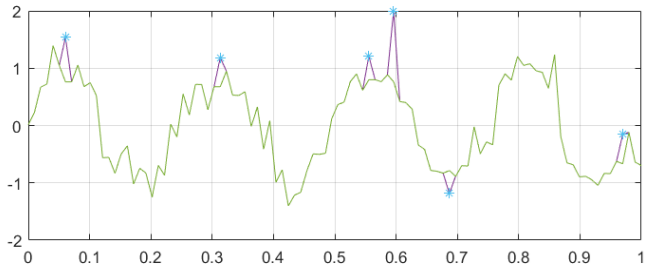
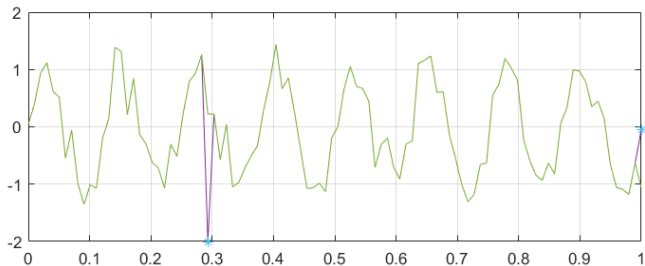
Notez que  $\gamma_k = \text{median}(x_{n-k}, \dots, x_{n+k})$  et  $\sigma_k = \zeta \times \text{median}(|x_{n-k} - \gamma_k|, \dots, |x_{n+k} - \gamma_k|)$  avec  $\zeta \approx 1.4826$  ( $\sigma_k/\zeta$  est la médiane de l'écart en valeur absolue).

```
clear;close all;clc
N=100;Duration=1;t=(0:Duration/(N-1):Duration);
x = sin(2*pi*[8;4]*t)'+0.3*randn(N,2);x(30,1)=-2;x(60,2)=2;
nsig = 2;% 2 x sigma
halfWindsize=4;%Wind size = 2 x k +1;
[y,h] = hampel(x,halfWindsize,nsig);
subplot(211);plot(t,x(:,1),t,y(:,1));grid
hold on;plot(t(h(:,1)),x(h(:,1),1),'*')
subplot(212);plot(t,x(:,2),t,y(:,2));grid
hold on;plot(t(h(:,2)),x(h(:,2),2),'*')
```

**D'autres scripts** : isoutlier, filloutliers

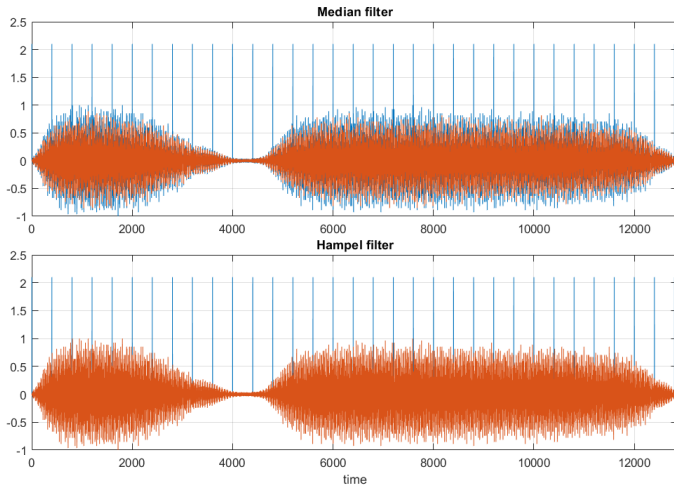


## c) Élimination des outliers



## c) Élimination des outliers

```
load train;y(1:400:end) = 2.1;u=medfilt1(y,3);v=hampel(y,13);
subplot(211);plot(y);hold on;plot(u);subplot(212);plot(y);hold on;plot(v);
```



## d) Filtrage/Lissage via décomposition

## d) Décomposition dans une base de signaux

Soit la décomposition suivante :

$$p(n) = \sum_{m=1}^M w_m b_m(n)$$

$$p(n) = b_1(n)w_1 + b_2(n)w_2 + \dots + b_M(n)w_M$$

En utilisant seulement 3 atomes  $\forall t \geq 0$ , il vient :

$$\begin{pmatrix} p(0) \\ \vdots \\ p(1) \\ \vdots \\ p(N) \end{pmatrix} = \begin{pmatrix} b_1(0) & b_2(0) & b_3(0) \\ \vdots & \vdots & \vdots \\ b_1(1) & b_2(1) & b_3(1) \\ \vdots & \vdots & \vdots \\ b_1(N) & b_2(N) & b_3(N) \end{pmatrix} \begin{pmatrix} w_1 \\ w_2 \\ w_3 \end{pmatrix}$$

En proposant différentes valeurs à  $p(n)$ , il vient :

$$\mathbf{p} = \mathbf{B}\mathbf{w}$$

Supposons  $p(n)$  une mesure et  $b_m(n)$  une base connue de fonctions.

Comment trouver les paramètres  $w_m$  en vue de fabriquer un signal artificiel  $\tilde{p}(n)$  aussi proche que possible de  $p(n)$  ?

En supposant  $\mathbf{p}$  comme un vecteur de longueur finie et  $\mathbf{B}$  comme une matrice carrée de dimension finie, le vecteur de paramètres peut être obtenu par :

$$\mathbf{w} = \mathbf{B}^{-1}\mathbf{p}$$

Toutefois, comment calculer  $\mathbf{w}$  lorsque :

- 1  $\mathbf{p}$  n'est pas un vecteur de longueur finie;
- 2  $\mathbf{B}$  n'est pas une matrice carrée.

## d) Solution

$$\mathbf{B}^T \mathbf{p} = \mathbf{B}^T \mathbf{B} \mathbf{w}$$

$$(\mathbf{B}^T \mathbf{p}) = (\mathbf{B}^T \mathbf{B}) \mathbf{w}$$

$$\mathbf{g} = \mathbf{Q} \mathbf{w}$$

$$\begin{pmatrix} b_1(0) & \vdots & b_1(N) \\ b_2(0) & \vdots & b_2(N) \\ b_3(0) & \vdots & b_3(N) \end{pmatrix} \begin{pmatrix} p(0) \\ \vdots \\ p(N) \end{pmatrix} = \begin{pmatrix} b_1(0) & \vdots & b_1(N) \\ b_2(0) & \vdots & b_2(N) \\ b_3(0) & \vdots & b_3(N) \end{pmatrix} \begin{pmatrix} b_1(0) & b_2(0) & b_3(0) \\ \vdots & \vdots & \vdots \\ b_1(N) & b_2(N) & b_3(N) \end{pmatrix} \begin{pmatrix} w_1 \\ w_2 \\ w_3 \end{pmatrix}$$

La matrice  $\mathbf{Q}$  est carrée de dimension finie et la solution des moindres carrés conduit à :

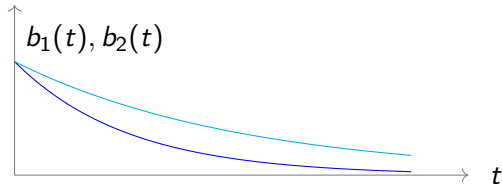
$$\min_{\mathbf{w}} \|\mathbf{p} - \mathbf{B} \mathbf{w}\|_2^2 \Rightarrow \mathbf{w} = (\mathbf{B}^T \mathbf{B})^{-1} \mathbf{B}^T \mathbf{p}.$$

$$\mathbf{w} = \mathbf{Q}^{-1} \mathbf{g}$$

**Question :** Soit le signal  $p(t) = \text{Rect}_1(t - 1/2)$ .

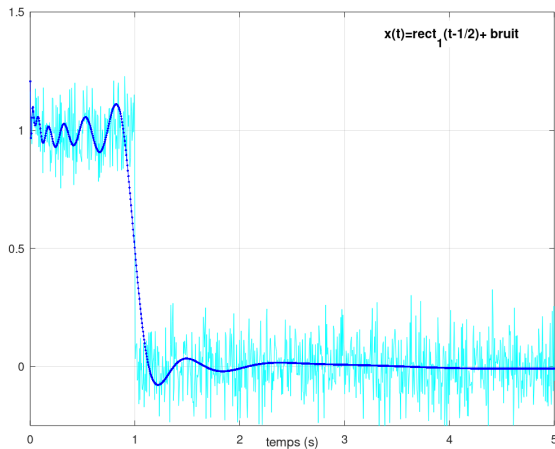


Décomposez  $p(t)$  dans une base exponentielle :  $b_m(t) = e^{-mt}\epsilon(t)$ ,  $\epsilon(t)$  est la fonction d'Heaviside.



## d) Débruitage signal porte

```
clear all;clc;close all;
N=1000;t=0:5/(N-1):5;M=30;
Fe=N/5;p=zeros(N,1);
p(1:200)=1;p=p+0.1*randn(N,1);
for l=1:M
    B(:,l)=exp(-l*t);
end
w=inv(B'*B)*(B'*p);
ptild=B*w;
plot(t,p,'c',t,ptild,'b.-');
```





**Objectif :** Trouver un polynôme qui ajuste au mieux les données bruitées en vue d'en extraire la tendance. Les données sont un dataset de  $N$  points:  $\mathbf{x} = \{t_1, t_2, \dots, t_N\}$  et  $\mathbf{y} = \{y_1, y_2, \dots, y_N\}$ . La solution est obtenue en minimisant l'énergie du résidu:

$$r = \sum_{n=1}^N (y_n - p(t_n))^2,$$

où  $p(t_n)$  est le polynôme de la forme :  $p(t_n) = a_0 + a_1 t_n + a_2 t_n^2$ . La solution peut être obtenue en résolvant le système d'équation surdéterminé (il y a plus d'équations que d'inconnues) :

$$\mathbf{y} = \mathbf{H}\mathbf{a}$$

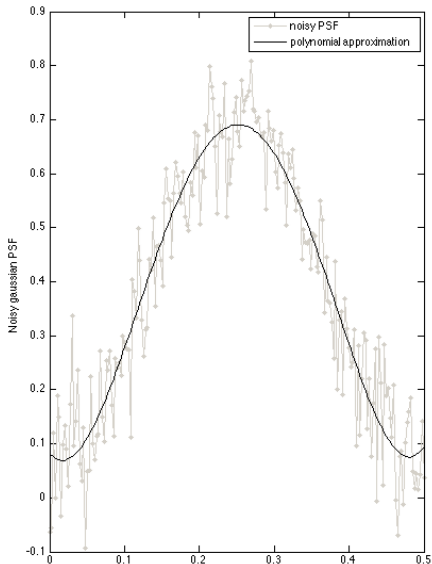
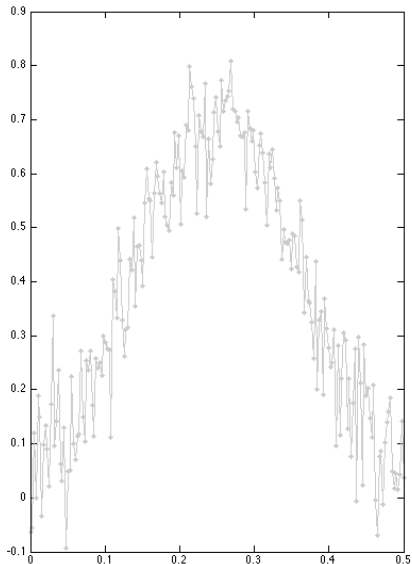
$$\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix} = \begin{pmatrix} 1 & t_1 & t_1^2 \\ 1 & t_2 & t_2^2 \\ \vdots & \vdots & \vdots \\ 1 & t_N & t_N^2 \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ a_2 \end{pmatrix}$$

L'énergie du résidu s'écrit :  $E = \|\mathbf{y} - \mathbf{H}\mathbf{a}\|_2^2$  et la solution est donnée par :

$$\mathbf{a} = (\mathbf{H}^T \mathbf{H})^{-1} \mathbf{H}^T \mathbf{y}.$$

```
clear;clc;close all;
N=200;tn=(0:N-1)';tno=N/2;sigma=N/5;C=1;
y=C+exp(-(tn-tno).^2/2/sigma^2);
ynoisyy=y+C/10*randn(N,1);
orderMax=2;
H = bsxfun(@power, tn, [orderMax:-1:0]);
p = (H'*H) \ (H'*ynoisyy);
yhat=polyval(p, tn);
figure(1);tx=0:0.5/(N-1):0.5;
plot(tx,y,'k--',tx,ynoisyy,'c.-',tx,yhat,'b');
```

## d) Approximation Polynomiale



Une autre approche possible pour débruiter des données bruitées consiste à utiliser la régularisation pondérée des moindres carrés<sup>2</sup>. Le signal lisse  $x(n)$  peut être obtenu comme solution au problème :  $\min_{\mathbf{x}} \|\mathbf{y} - \mathbf{x}\|_2^2 + \lambda \|\mathbf{D}\mathbf{x}\|_2^2$  avec

$$\mathbf{D} = \begin{pmatrix} 1 & -2 & 1 & & \\ & 1 & -2 & 1 & \\ & & \ddots & \ddots & \\ & & & 1 & -2 & 1 \end{pmatrix}$$

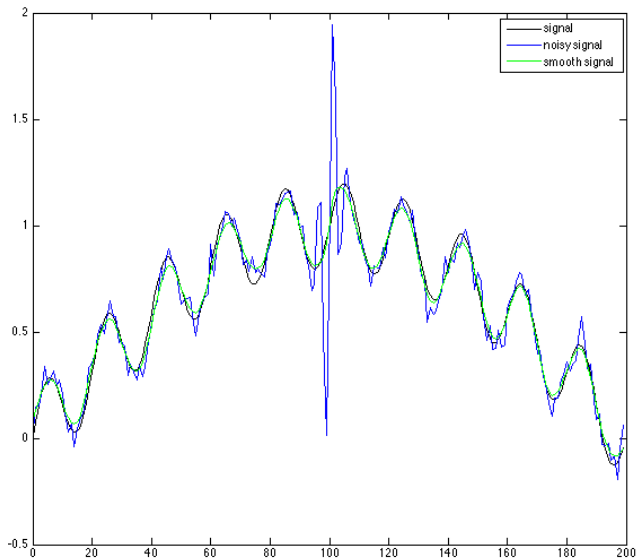
Le solution s'écrit :  $\mathbf{x} = (\mathbf{I} + \lambda \mathbf{D}^T \mathbf{D})^{-1} \mathbf{y}$ .

Minimiser  $\|\mathbf{y} - \mathbf{x}\|_2^2$  force  $\mathbf{x}$  à être similaire au signal bruité  $\mathbf{y}$  et minimiser  $\|\mathbf{D}\mathbf{x}\|_2^2$  force  $\mathbf{x}$  à être lisse.

---

<sup>2</sup>La douceur d'un signal peut être mesurée par l'énergie de sa dérivée (ou dérivée d'ordre  $n$ ).

## d) Lissage / Smoothing

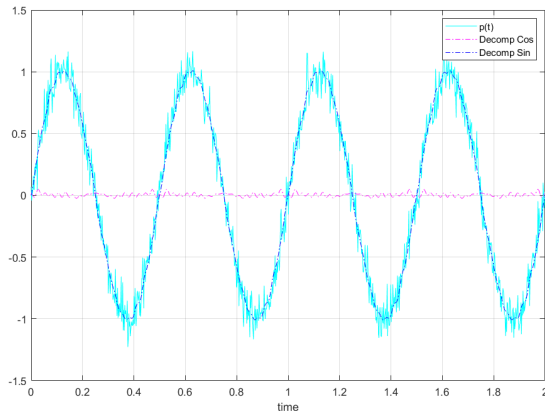


### MATLAB CODE (Ivan Selesnick, 2013):

```
clear;clc;close all;
N=200;n=(0:N-1)';no=N/2;sigma=N/80;fo=5;
xx=0.2*sin(4*pi*fo*n/N)+sin(pi*fo/5*n/N);
Noise=0.05*randn(N,1)+(sin(2*pi*8*fo*n/N)...
.*exp(-(n-no).^2/2/sigma^2));
y=xx+Noise;
order=2;e = ones(N, 1);
D = spdiags([e -2*e e], 0:2, N-2, N);
lam = 20;
F = speye(N) + lam * D' * D;
x = F \ y;
figure(1);clf;
plot(n,xx,'k',n,y,'b',n,x,'g');
title('Data');
legend('signal','noisy signal','smooth signal');
```

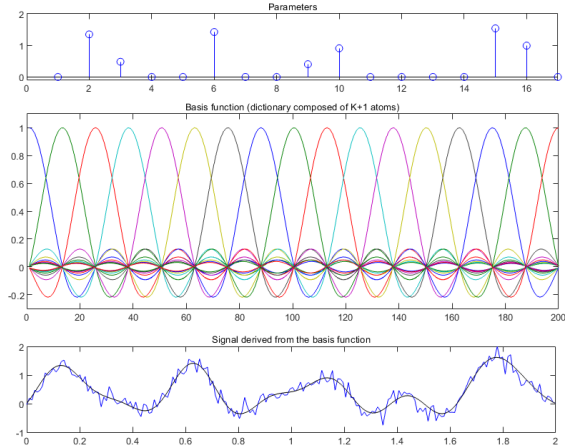
## d) Débruitage de sinusôides

```
clear all;clc;close all;
N=1000;t=0:2/(N-1):2;
T0=0.5;f0=1/T0;
bruit=0.1*randn(N,1);
p=sin(2*pi*1/T0*t)'+bruit;
K=20;
for k=1:K;
    B1(:,k)=cos(2*pi*f0*(k-1).*t);
    B2(:,k)=sin(2*pi*f0*(k-1).*t);
end
w1=pinv(B1)*p;ptild1=B1*w1;
w2=pinv(B2)*p;ptild2=B2*w2;
plot(t,p,t,ptild1,t,ptild2);
```



## d) Débruitage via Base de Dirichlet

Soit un dictionnaire (Base de Dirichlet)  $\phi_k(n) = \text{sinc}(\pi f_0(t - kt_0))$  ici de  $K = 16$  atomes.





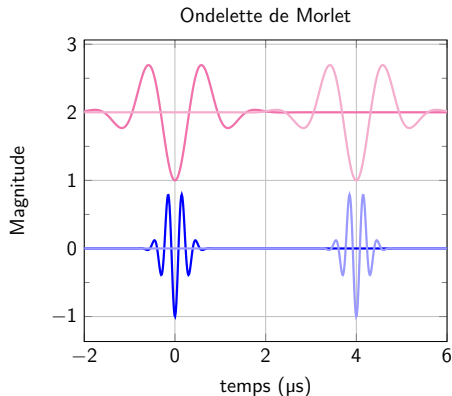
### Les ondelettes :

$$\psi_{a,b}(t) = \frac{1}{\sqrt{a}} \psi \left( \frac{t-b}{a} \right)$$

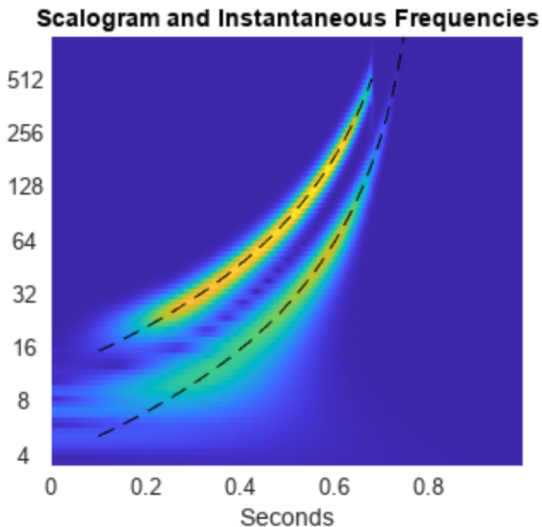
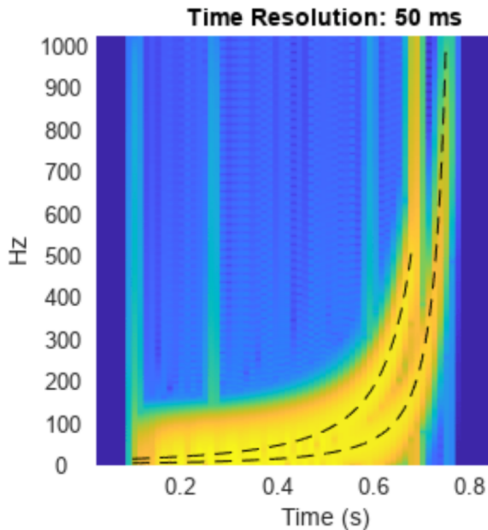
où  $a > 0$  est un facteur de dilatation (ou échelle) et  $b$  est un facteur de translation. La transformée en ondelettes continues de  $x(t)$  est définie par :

$$W(a, b) = \int_{-\infty}^{\infty} x(t) \psi_{a,b}^*(t) dt$$

La transformée en ondelette produit un spectre "temps-échelle".



## d) Scalogram



Le nombre d'atomes est limité au nombre d'échelles et nombre de translation avec  $a = 2^j$  et  $b = k 2^j$  avec  $J$  le niveau d'approximation de la décomposition.

$$x(t) = A_J(t) + \sum_{j=1}^J D_j(t),$$

où  $A_J(t) = x(t) * h_{lp}^{(J)}(t)$  capture les tendances globales à la plus basse fréquence et

$D_j(t) = x(t) * h_{hp}^{(j)}(t)$  les détails à chaque échelle  $j$ . Exemple avec  $J = 2$  :

$x(t) = A_2(t) + D_1(t) + D_2(t)$ . Les bandes de fréquence sont définies<sup>3</sup> :

Pour  $A_2 : [0, f_s/4]$ , pour  $D_1 : [f_s/4, f_s/2]$  et  $D_2 : [f_s/2, f_s]$

---

<sup>3</sup> $A_j : [0, f_s/2^j]$  et  $D_j : [f_s/2^{j+1}, f_s/2^j]$

Dans le cas discret on a :

$$x(n) = A_J(n) + \sum_{j=1}^J D_j(n).$$

Pour un seul niveau de décomposition  $J = 1$  :

$$A_1(n) = x(n) * h_{lp}^1(n) = \sum_k x(k) h_{lp}^1(n - k) \text{ et}$$

$$D_1(n) = x(n) * h_{hp}^1(n) = \sum_k x(k) h_{hp}^1(n - k).$$

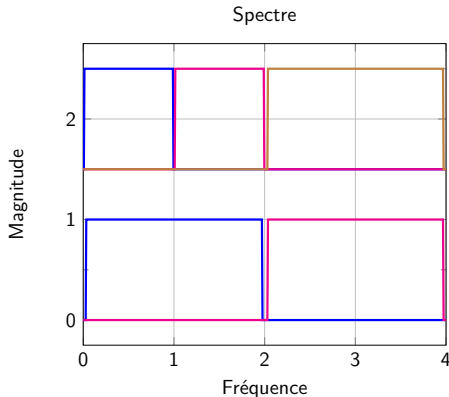
Pour deux niveaux de décomposition  $J = 2$  :

$$A_2(n) = x(n) * h_{lp}^2(n), D_1(n) = x(n) * h_{hp}^1(n) \text{ et}$$

$$D_2(n) = x(n) * h_{hp}^2(n).$$

Pour une décomposition de Haar,  $h_{lp}^1 = [1, 1]/\sqrt{2}$

et  $h_{hp}^1 = [1, -1]/\sqrt{2}$ .

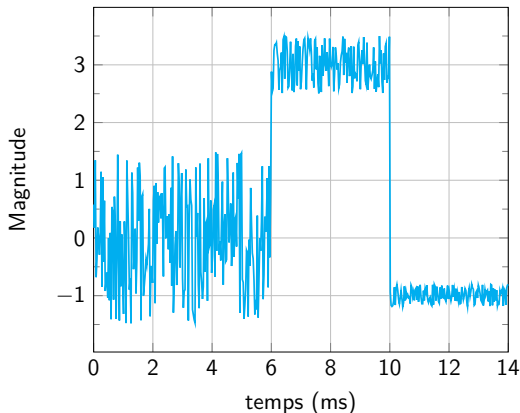


## d) Débruitage : seuillage adaptatif

Ici on suppose une observation  $y(t)$  issue d'un signal  $x(t)$  corrompu par un bruit dont le niveau qui varie avec le temps  $\sigma(t)$ , le modèle s'écrit :  $y(t) = x(t) + b(t)$ .

Un premier niveau de simplification est de supposer que la variance du bruit varie par morceaux, mais les intervalles et les valeurs de la variance sont inconnus.

Le problème revient à identifier les points de passage d'un segment à l'autre. Pour se faire on réalise un filtre passe-haut pour récupérer exclusivement le bruit. Ensuite il suffit d'estimer les positions de changement de variance.

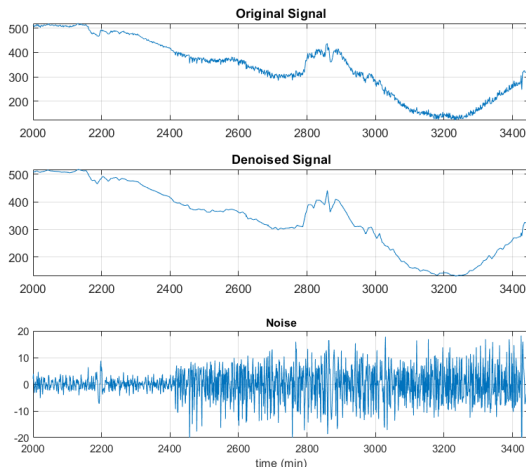


Soit un signal issu d'un système électrique ayant subi une défaillance à 2410 min.

Le bruit est non blanc.

```
load leleccum
indx = 2000:3450; x = leleccum(indx);
xd = wdenoise(x,3,'Wavelet','db3',...
'DenoisingMethod','UniversalThreshold',...
'ThresholdRule','Soft',...
'NoiseEstimate','LevelDependent');
```

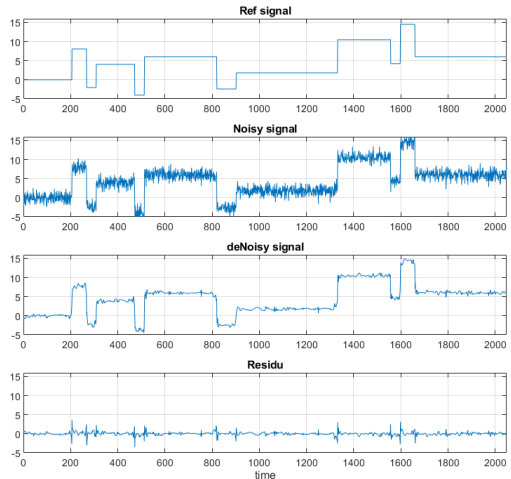
Débruitage réalisé avec 3 niveaux d'échelle et une ondelette de type 'db3' avec un seuillage doux et une estimation du niveau du bruit avec la fonction 'LevelDependent' (à chaque niveau de résolution).



Soit un signal bruité possédant des paliers.

```
sqrt_snr = 4; init = 42;
[xref,x] = wnoise(1,11,sqrt_snr,init);
xd = wdenoise(x, 'Wavelet', 'sym8', ...
    'DenoisingMethod', 'SURE', ...
    'ThresholdRule', 'Soft');
```

Débruitage réalisé avec 8 niveaux d'échelle et une ondelette de type 'sym' avec un seuillage heuristique SURE doux opérant sur les coefficients de détails.



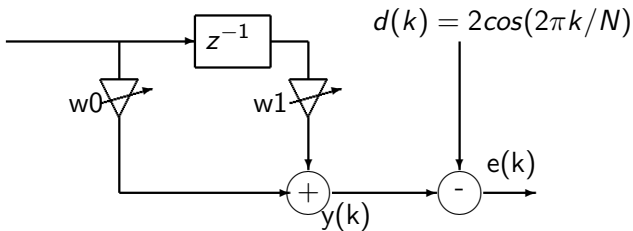
## e) Filtrage Optimal de Wiener



## e) Combineur linéaire adaptatif

Déterminer les paramètres  $w_0$  et  $w_1$  qui permettent de fabriquer un signal cosinus  $d(k)$  (le signal désiré) à partir d'un signal sinus  $x(k)$  de  $N$  échantillons par cycle. En d'autres termes, calculer les paramètres qui procurent un déphasage de 90 degrés avec  $y(k) = w_0x(k) + w_1x(k-1)$ .

$$x(k) = \sin(2\pi k/N)$$



Le signal  $y(k)$  est obtenu par filtrage du signal  $x(k)$  :  $y(k) = \sum_{l=0}^L w_l x(k-l)$ , qui se réduit dans notre cas à  $y(k) = w_0 x(k) + w_1 x(k-1)$ .

En notation vectorielle, le signal  $y(k)$  s'écrit :

$$y(k) = \mathbf{H}^T(k) \mathbf{W} = \mathbf{W}^T \mathbf{H}(k),$$

$\mathbf{W} = [w_0, w_1]^T$  et  $\mathbf{H}(k) = [x(k), x(k-1)]^T$ .

Sous hypothèse que  $e(k)$ ,  $d(k)$  et  $\mathbf{H}(k)$  sont stationnaires, la fonction performance à minimiser est :

$$J(\mathbf{W}) = E[e^2(k)] = E[(d(k) - y(k))^2]$$

avec

$$e(k) = d(k) - \mathbf{W}^T \mathbf{H}(k).$$

$$e^2(k) = \left( d(k) - \mathbf{W}^T \mathbf{H}(k) \right) \left( d(k) - \mathbf{W}^T \mathbf{H}(k) \right)^T.$$

$$e^2(k) = d^2(k) + \mathbf{W}^T \mathbf{H}(k) \mathbf{H}^T(k) \mathbf{W} - 2d(k) \mathbf{H}^T(k) \mathbf{W}(k)$$

La fonction de coût à minimiser s'écrit :

$$E[e^2(k)] = E[d^2(k)] + \mathbf{W}^T E[\mathbf{H}(k) \mathbf{H}^T(k)] \mathbf{W} - 2E[d(k) \mathbf{H}^T(k)] \mathbf{W}.$$

En posant  $\mathbf{R} = E[\mathbf{H}(k) \mathbf{H}^T(k)]$  et  $\mathbf{P} = E[d(k) \mathbf{H}(k)]$ , l'erreur quadratique moyenne (EQM) s'écrit :

$$\xi = E[e^2(k)] = E[d^2(k)] + \mathbf{W}^T \mathbf{R} \mathbf{W} - 2\mathbf{P}^T \mathbf{W}.$$

Minimiser l'EQM par rapport aux coefficients optimaux recherchés  $\mathbf{W}^*$  revient à calculer pour chaque valeur de  $k$  :

$$\nabla(\xi) = \frac{d\xi}{d\mathbf{W}} = 0.$$

$$\frac{d\xi}{d\mathbf{W}} = \left[ \frac{d\xi}{dw_0}, \frac{d\xi}{dw_1}, \dots, \frac{d\xi}{dw_L} \right]^T$$

Sachant que  $\frac{d(\mathbf{A}^T \mathbf{H})}{d\mathbf{H}} = \mathbf{A}$  et que  $\frac{d(\mathbf{H}^T \mathbf{A} \mathbf{H})}{d\mathbf{X}} = 2\mathbf{A} \mathbf{H}$  alors (voir page 76)

$$\frac{d\xi}{d\mathbf{W}} = 2\mathbf{R} \mathbf{W} - 2\mathbf{P},$$

$$\nabla(\xi) = 0 = 2\mathbf{R} \mathbf{W}^* - 2\mathbf{P},$$

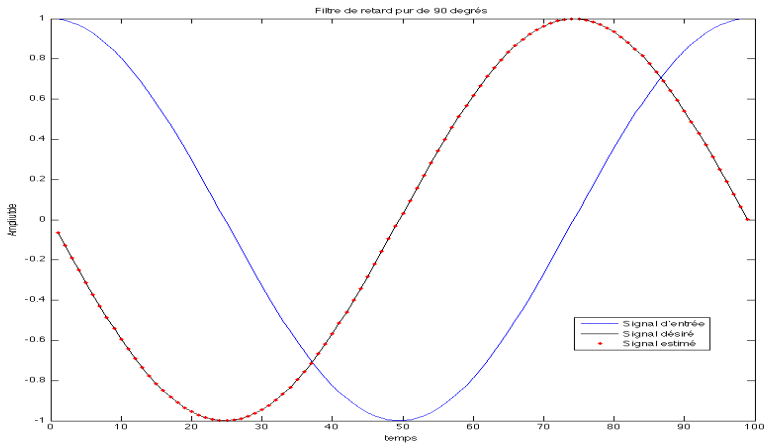
$$\mathbf{R} \mathbf{W}^* = \mathbf{P},$$

$$\mathbf{W}^* = \mathbf{R}^{-1} \mathbf{P}.$$

```

clear;clc;close all
N=100;Fe=100;Te=1/Fe;n=0:10/(N-1):10;fo=10;
x=cos(2*pi*fo*n*Te)';theta=pi/2;
d=cos(2*pi*fo*n*Te+theta)';
H=[ ];p=2;
for k=p:N;
    H=[H;x(k) x(k-1)];
end
R=H'*H;
P=H'*d(p:end,1);
W=inv(R)*P;
Y=H*W;
W =[15.74, -15.77]
figure
plot(x(p:end));hold on;plot(d(p:end),'k');hold on;plot(Y,'r.')
legend('Signal d\'entree', 'Signal desire', 'Signal estime')
xlabel('temps');ylabel('Amplutude');title('Filtre de retard pur de 90 ...
degres')
  
```

## e) Filtre déphaseur



*Filtre déphaseur de  $\pi/2$ .*

## e) Remarque : Orthogonalité

Remarquons que lorsque la réponse impulsionnelle du filtre est optimale  $\mathbf{W} = \mathbf{W}^*$ , le signal erreur  $e(k)$  est complètement décorrélé ou orthogonal au signal d'entrée  $x(k)$ .

En effet :

$$e(n) = d(k) - \mathbf{H}^T(k)\mathbf{W}.$$

En multipliant l'expression précédente par  $\mathbf{H}(k)$ , il vient :

$$e(n)\mathbf{H}(k) = d(k)\mathbf{H}(k) - \mathbf{H}(k)\mathbf{H}^T(k)\mathbf{W}.$$

En prenant l'espérance mathématique de l'expression précédente, il vient :

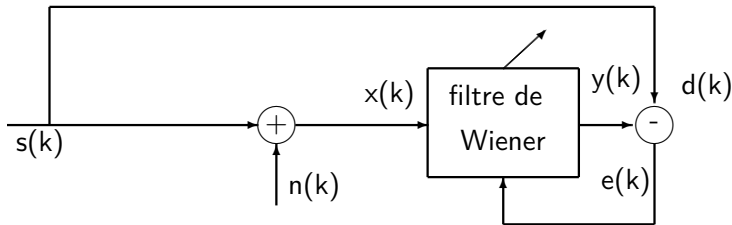
$$E[e(n)\mathbf{H}(k)] = \mathbf{P} - \mathbf{R}\mathbf{W}.$$

Lorsque le filtre est optimal,  $\mathbf{W} = \mathbf{W}^* = \mathbf{R}^{-1}\mathbf{P}$  et il vient :

$$E[e(n)\mathbf{H}(k)]_{\mathbf{W}=\mathbf{W}^*} = \mathbf{P} - \mathbf{P} = 0.$$

## e) Filtre débruiteur de Wiener

On dispose du signal d'entrée  $s(k)$  et d'une observation  $x(k)$  bruitée. L'objectif est de filtrer  $x(k)$  afin d'obtenir un signal  $d(k) = s(k)$ .



Pour ce faire, nous allons fabriquer un signal  $y(k) = \hat{d}(k)$  qui sera une estimation de  $d(k)$ . Ce signal  $y(k)$  sera la sortie d'un filtre optimal dont l'entrée sera l'observation  $x(k)$ . Le problème revient alors à calculer les paramètres  $w_l = w(l)$  du filtre qui minimise une fonction de coût.



## e) Filtre RIF de Wiener :

La sortie du filtre linéaire RIF de Wiener s'écrit :

$$y(k) = \sum_{l=0}^{L-1} w(l)x(k-l) = \mathbf{W}^T \mathbf{H}(k),$$

avec  $\mathbf{W} = [w_0, w_1, \dots, w_L]^T$  et  $\mathbf{H}(k) = [x(k), x(k-1), \dots, x(k-L)]^T$ .

La fonction de coût à minimiser est l'erreur quadratique moyenne (EQM):  $J(\mathbf{W}) = E[e^2(k)]$  où le signal erreur sera défini par :  $e(k) = d(k) - y(k)$ . La minimisation de l'EQM conduit à l'équation de **Wiener-Hopf** :

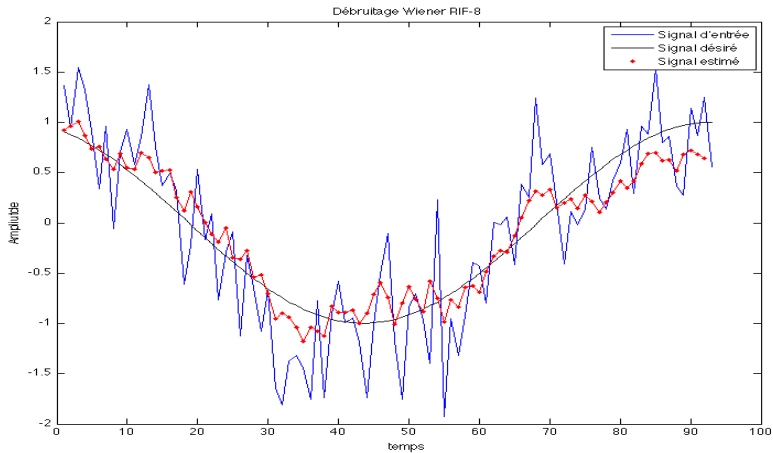
$$\frac{dJ(\mathbf{W})}{d\mathbf{W}} = 2\mathbf{P}\mathbf{W} - 2\mathbf{P} = 0,$$

$$\nabla(\xi) = 0 = 2\mathbf{R}\mathbf{W}^* - 2\mathbf{P},$$

$$\boxed{\mathbf{W}^* = \mathbf{R}^{-1}\mathbf{P}}.$$

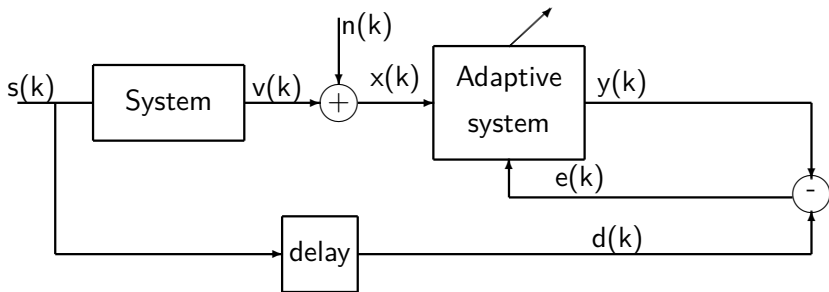
```
clear;clc;close all
N=100;Fe=100;Te=1/Fe; n=(0:20/(N-1):20)';fo=20;
b=0.5*randn(N,1);s=cos(2*pi*fo*n*Te);d=s;
x=s+b;
H=[];ord=8;
for k=ord:N;
H=[H,x(k:-1:k-ord+1)];
end
w=pinv(H')*d(ord:end,1);
y=H'*w;
figure;plot(x(ord:end));hold on;
plot(d(ord:end),'k');
plot(y,'r.-');
```

## e) Débruitage RIF-8 de Wiener



*Débruitage par filtre RIF de 8 coefs.*

## Structure du filtre égaliseur de Wiener :



```

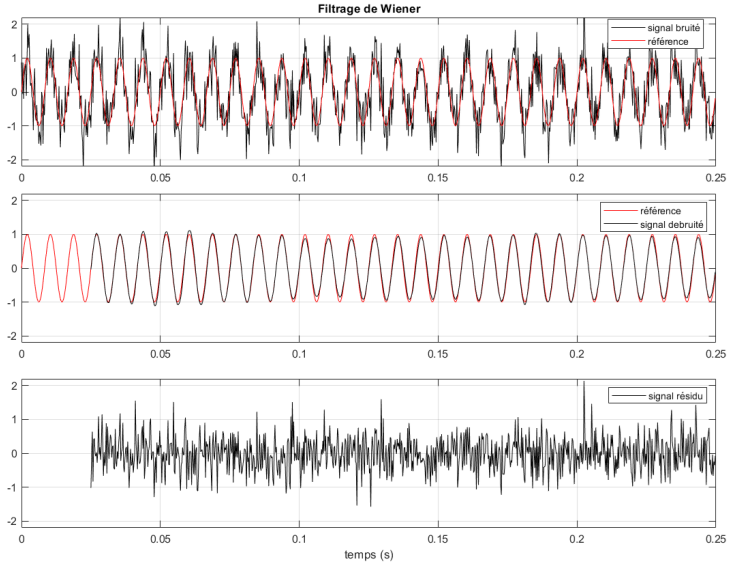
clear;clc;close all;
nbsymb = 200;sk = sign(randn(nbsymb, 1));
nk=0.1*randn(nbsymb, 1); numerator = [0; 1; 0.5]; denominator=1;
vk = filter(numerator, denominator, sk); xk=vk+nk;delay = 3;
mu = 0.05;m = 10; w = zeros(m, 1);W = zeros(m, nbsymb);
for k=1:(nbsymb-m+1)
    n = k+m-1; Xk = xk(n:-1:n-m+1); yk(n) = Xk' * w;
    dk(n)=sk(n-delay);Esy(n) = dk(n)-yk(n);
    w = w + mu * Xk * Esy(n); W(:, n) = w;
end;
Nk=(1:length(xk));
subplot(511);plot(Nk,xk,'b');ylim([-1.7 1.7]);
subplot(512);plot(Nk,dk,'k',Nk,yk,'m');
ylim([-1.7 1.7]);subplot(513);
plot(Nk,dk,'k',Nk,sign(yk),'r');ylim([-1.7 1.7]);
subplot(514);plot(W','k');ylim([-0.6 1.1]);
subplot(515);plot(Nk,Esy,'b',Nk,dk-sign(yk),'r');
ylim([-1.1 1.1]);
  
```

```
%written by M. Buzzoni, May 2019
```

```
N=1000;Duration=0.25;f0=120;t=(0:Duration/(N-1):Duration)';
ref = sin(2*pi*f0*t);b=0.50*randn(N,1);x = ref + b;Order=100;
[xest,b,MSE] = wienerFilt(x,ref,Order);
subplot(311);plot(t,x,'k',t,ref,'r')
subplot(312);plot(t,ref,'r',t(Order+1:end),xest,'k');
subplot(313);plot(t(Order+1:end),(x(Order+1:end) - xest),'k')
```

```
function [xest,B,MSE] = wienerFilt(x,y,N)
%written by M. Buzzoni, May 2019
X = 1/N .* fft(x(1:N));Y = 1/N .* fft(y(1:N));
X = X(:);Y = Y(:);
Rxx = N .* real(ifft(X .* conj(X))); % Autocorrelation function
Rxy = N .* real(ifft(X .* conj(Y))); % Crosscorrelation function
Rxx = toeplitz(Rxx);Rxy = Rxy';
B = Rxy / Rxx; B = B(:);
xest = fftfilt(B,x);xest = xest(N+1:end);
MSE = mean(y(N+1:end) - xest) .^2;
```

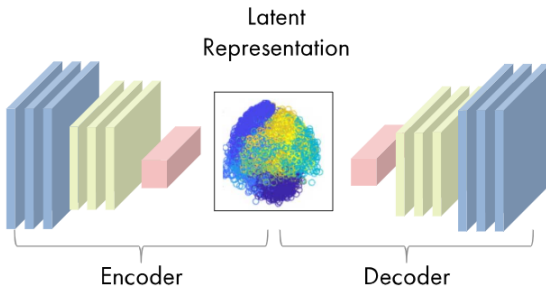
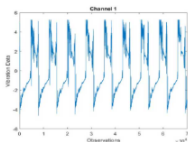
## e) Wiener fréquentiel



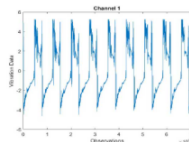
# Bonus 1 : Auto Encoder



Input – Normal Operating Data



Output signal



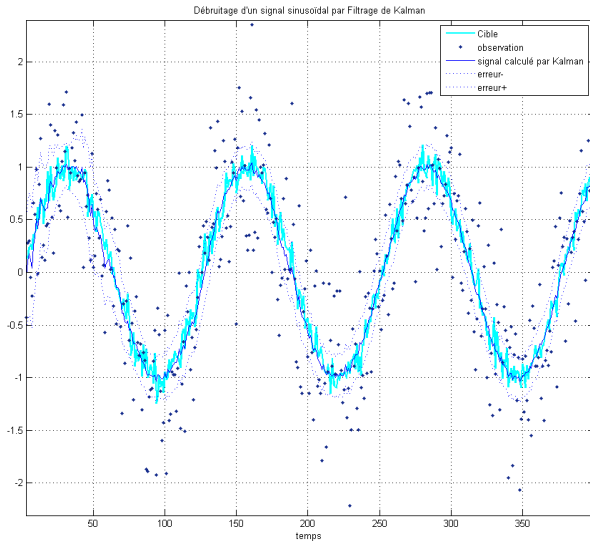
# Bonus 2 : Filtrage de Kalman

```
% Debruitage d'une sinusoide
clear, clc; close all; N=100; t=0:N-1;
% x(k+1)=A x(k)+ w(k+1);
% y(k)      =C x(k)+ v(k);
s.G=1; s.Rw = 4; s.Rv =4; wk=sqrt(s.Rw)*randn(1,N);
vk=sqrt(s.Rv)*randn(1,N); s.A = 1; s.C = 1; s.B = 0; s.u = 0;
xk=8*cos(2*pi*3*(t)/N); yk=s.C*xk+vk; s.x = nan; s.P = nan;
for t=1:N-1
    s(1).y=yk(t);
    s=KalmanFilter(s);
    xfk(t+1)=s.x;
    Gk(t+1)=s.G;
end
figure; hz=plt(yk, 'r. '); hold on; hk=plot(xfk, 'b- '); ht=plot(xk, 'g- ');
legend([hz hk ht], 'Observation', 'Sortie Kalman', 'Signal original', 0)
figure; plot(Gk); title('Gain de Kalman')
```

```

function s = KalmanFilter(s)
%  $x = Ax + Bu + w$ 
%  $y = Cx + v$ 
if isnan(s.x)
% Utile seulement pour la premiere iteration
s.x = inv(s.C)*s.y;
s.P = inv(s.C)*s.Rv*inv(s.C');
else
% Code du filtre de Kalman :
% Prediction du vecteur d'etat et de la matrice de covariance P :
s.x = s.A*s.x + s.B*s.u;
s.P = s.A * s.P * s.A' + s.Rw;
% Calcul du Gain de Kalman :
G = s.P*s.C'*inv(s.C*s.P*s.C'+s.Rv);
% Correction basee sur l'observation:
s.x = s.x + G*(s.y-s.C*s.x);
s.P = s.P - G*s.C*s.P;
s.G=G;
end
  
```

# Kalman : Débruitage d'un signal sinusoïdal



# Bonus 3 : Filtres parcimonieux

Go back to the under-determined system of equations:

$$\mathbf{y} = \mathbf{Ax}.$$

There is another way to solve  $\mathbf{y} = \mathbf{Ax}$ . It consists to minimize the sum of absolute values of  $\mathbf{x}$ :

$$\arg \min_{\mathbf{x}} \|\mathbf{x}\|_1 \quad \text{s.t.} \quad \mathbf{y} = \mathbf{Ax}.$$

This problem known as the *basis pursuit* (BP) problem can not be written in explicit form. The solution can be found in a iterative way.

When  $\mathbf{y}$  is noisy, the best way is to find an approximation by minimizing the cost function:

$$\arg \min_{\mathbf{x}} \|\mathbf{y} - \mathbf{Ax}\|_2^2 + \lambda \|\mathbf{x}\|_1.$$

The solution is known as the *basis pursuit denoising* (BPD) problem. When  $\mathbf{x}$  has sparse properties, it is advantageous to use the  $L_1$  norm.

- BP and BPD cannot have an explicit form because  $\|\mathbf{x}\|_1$  is not differentiable (specifically at zero);
- However they can be solved iteratively with simple algorithm because BP and BPD are convex optimization problems (no local extrema);
- a certain number of iterative algorithms were developed to find a solution:
  - ▶ ISTA (Iterative Shrinkage Thresholding Algorithm);
  - ▶ FISTA (Fast ISTA);
  - ▶ SALSA (Split Variable Augmented Lagrangian Shrinkage Algorithm).



Consider a noisy signal  $\mathbf{y}$  to be denoised with the following cost function:

$$J(\mathbf{x}) = \|\mathbf{y} - \mathbf{x}\|_2^2 + \lambda \|\mathbf{D}\mathbf{x}\|_1,$$

where  $\|\mathbf{D}\mathbf{x}\|_1 = \sum_{n=2}^N |x(n) - x(n-1)|$  and where

$$\mathbf{D} = \begin{pmatrix} -1 & 1 & & & & \\ & & -1 & 1 & & \\ & \ddots & & & \ddots & \\ & & & & & -1 & 1 \end{pmatrix}$$

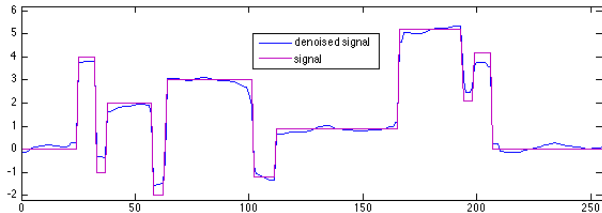
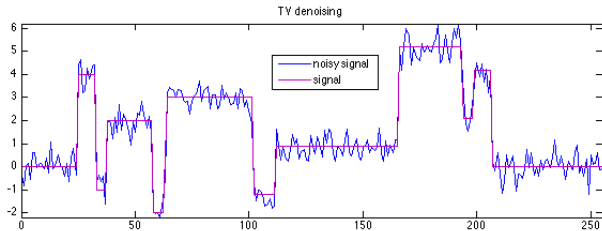
$$\mathbf{x}^{(i+1)} = \mathbf{y} - \mathbf{D}^T \mathbf{z}^{(i)}$$

$$\mathbf{z}^{(i+1)} = \text{clip} \left( \mathbf{z}^{(i)} + \frac{1}{\alpha} \mathbf{D}^T \mathbf{x}^{(i+1)}, \frac{\lambda}{2} \right)$$

## MATLAB CODE (Ivan Selesnick, 2013):

```
clear;clc;close all;;N = 256;
load signal; % signal is the signal
y = signal + 0.5*randn(N,1);y = y(:)';
Nit=20;lam=2;alpha = 3;T = lam/2;
J = zeros(1,Nit);z = zeros(1,N-1);
for k = 1:Nit
    x = y - [-z(1) -diff(z) z(end)];
    J(k) = sum(abs(x-y).^2) + lam * sum(abs(diff(x)));
    z = z + 1/alpha * diff(x);
    z = max(min(z,T),-T); % clip(z,T)
end
figure(1);clf;
subplot(2,1,1);plot(0:N-1,y,'-',0:N-1,signal,'r');
title('TV denoising');axis([0 N-1 -2.2 6.2]);
legend('noisy signal','signal');
subplot(2,1,2);plot(0:N-1,x,'-',0:N-1,signal,'r');
axis([0 N-1 -2.2 6.2]);
```

# Total Variation Denoising



# Bonus 4 : Dérivée Gradient

What is the derivative of  $J(\mathbf{x}) = \mathbf{x}^T \mathbf{A} \mathbf{x}$  knowing that  $\frac{dJ(\mathbf{x})}{d\mathbf{x}} = \lim_{h \rightarrow 0} \frac{J(\mathbf{x}+h) - J(\mathbf{x})}{h}$ .

$$J(\mathbf{x} + \mathbf{h}) = (\mathbf{x} + \mathbf{h})^T \mathbf{A} (\mathbf{x} + \mathbf{h}) = \mathbf{x}^T \mathbf{A} \mathbf{x} + \mathbf{h}^T \mathbf{A} \mathbf{x} + \mathbf{x}^T \mathbf{A} \mathbf{h} + \mathbf{h}^T \mathbf{A} \mathbf{h}$$

$$J(\mathbf{x} + \mathbf{h}) - J(\mathbf{x}) = \mathbf{h}^T \mathbf{A} \mathbf{x} + \mathbf{x}^T \mathbf{A} \mathbf{h} + o(\mathbf{h})$$

with  $\mathbf{h}^T \mathbf{A} \mathbf{h} = o(\mathbf{h}) \approx 0$ .

Knowing that  $\mathbf{h}^T \mathbf{A} \mathbf{x}$  is a scalar, then  $\mathbf{h}^T \mathbf{A} \mathbf{x} = (\mathbf{h}^T \mathbf{A} \mathbf{x})^T = \mathbf{x}^T \mathbf{A}^T \mathbf{h}$ .

$$J(\mathbf{x} + \mathbf{h}) - J(\mathbf{x}) = \mathbf{x}^T \mathbf{A}^T \mathbf{h} + \mathbf{x}^T \mathbf{A} \mathbf{h}$$

$$J(\mathbf{x} + \mathbf{h}) - J(\mathbf{x}) = (\mathbf{x}^T \mathbf{A}^T + \mathbf{x}^T \mathbf{A}) \mathbf{h}$$

$$\frac{d(\mathbf{x}^T \mathbf{A} \mathbf{x})}{d\mathbf{x}} = \frac{J(\mathbf{x} + \mathbf{h}) - J(\mathbf{x})}{\mathbf{h}} = \mathbf{x}^T (\mathbf{A}^T + \mathbf{A}).$$

If the matrix  $\mathbf{A}$  is symmetric, then  $\mathbf{A}^T = \mathbf{A}$  and  $\frac{d(\mathbf{x}^T \mathbf{A} \mathbf{x})}{d\mathbf{x}} = \mathbf{x}^T (\mathbf{A} + \mathbf{A}) = 2\mathbf{x}^T \mathbf{A}$ . If  $\mathbf{A} = \mathbf{I}$  then  $\frac{d(\mathbf{x}^T \mathbf{I} \mathbf{x})}{d\mathbf{x}} = 2\mathbf{x}^T$ . (back to page 51)